

EXPERIMENT – 2

Develop and compare CLI, GUI, and VUI for the same task and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

Roll No. 240701253

Name: Keerthana C

AIM:

The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

PROCEDURE:

i) CLI (Command Line Interface)

A command-line voice assistant that accepts text input and generates speech responses using pyttsx3. It supports commands to check time, open websites (YouTube/Google), launch Notepad, and exit the application.

```
import pyttsx3

import datetime

import webbrowser

import os

engine = pyttsx3.init()
```

```
def speak(text):

    print("Assistant:", text)

    engine.say(text)

    engine.runAndWait()


def process_command(text):

    text = text.lower()

    if "time" in text:

        now = datetime.datetime.now().strftime("%H:%M:%S")

        speak("The time is " + now)

    elif "open youtube" in text:

        speak("Opening YouTube")

        webbrowser.open("https://youtube.com")

    elif "open google" in text:

        speak("Opening Google")

        webbrowser.open("https://google.com")

    elif "open note" in text:

        speak("Opening Notepad")
```

```
os.system("notepad")

elif "exit" in text:

    speak("Goodbye")

    return False

else:

    speak("Command not recognized")

return True

speak("Assistant started. Type commands.")

running = True

while running:

    command = input("You: ")

    running = process_command(command)
```

OUTPUT:

```
PS D:\projects> python cli2.py
Assistant: Assistant started. Type commands.
You: open google
Assistant: Opening Google
You: open youtube
Assistant: Opening YouTube
You: open notepad
Assistant: Opening Notepad
You: time
Assistant: The time is 16:14:30
You: exit
Assistant: Goodbye
```

ii) GUI (Graphical User Interface)

A GUI voice assistant built with Tkinter that accepts text commands to perform tasks like checking time, searching web, opening/closing Notepad, and writing text, with voice responses and automated interactions using pyautogui. It features a simple interface with command input, execution button, and color-coded output display.

```
import tkinter as tk

import pyttsx3

import datetime

import webbrowser

import subprocess

import pyautogui

import time
```

```
engine = pyttsx3.init()

opened_apps = {}

def speak(text):

    engine.say(text)

    engine.runAndWait()

def process_command():

    text = entry.get().lower()

    entry.delete(0, tk.END)

    if "time" in text:

        now = datetime.datetime.now().strftime("%H:%M:%S")

        output.config(text="Time: " + now, fg="blue")

        speak("The time is " + now)

    elif text.startswith("google"):

        query = text.replace("google", "").strip()

        output.config(text="Searching Google: " + query, fg="green")

        speak("Searching Google")
```

```
webbrowser.open("https://www.google.com")

time.sleep(3)

pyautogui.write(query)

pyautogui.press("enter")

elif text.startswith("youtube"):

    query = text.replace("youtube","").strip()

    output.config(text="Searching YouTube: " + query, fg="red")

    speak("Searching YouTube")

    webbrowser.open("https://www.youtube.com")

    time.sleep(4)

    pyautogui.write(query)

    pyautogui.press("enter")

elif text.startswith("write"):

    message = text.replace("write","").strip()

    output.config(text="Writing in Notepad", fg="purple")
```

```
speak("Opening Notepad")

opened_apps["notepad"] = subprocess.Popen("notepad.exe")

time.sleep(2)

pyautogui.write(message)

elif "open note" in text:

    output.config(text="Opening Notepad", fg="purple")

    speak("Opening Notepad")

    opened_apps["notepad"] = subprocess.Popen("notepad.exe")

elif "close note" in text or "close notepad" in text:

    if "notepad" in opened_apps:

        opened_apps["notepad"].terminate()

        output.config(text="Notepad Closed", fg="brown")

        speak("Notepad closed")

    else:

        output.config(text="Notepad not open", fg="orange")

elif text in ["exit", "quit", "bye", "close assistant"]:

    output.config(text="Assistant closing...", fg="black")
```

```
        speak("Goodbye")

        root.after(1200, root.destroy)

    else:

        output.config(text="Command not recognized", fg="orange")

        speak("Command not recognized")

root = tk.Tk()

root.title("Kikic Assistant")

root.geometry("450x260")

root.configure(bg="#e8f0ff")

title = tk.Label(root, text="Kikic Assistant", font=("Arial", 16, "bold"), bg="#e8f0ff")

title.pack(pady=10)

entry = tk.Entry(root, width=40, font=("Arial", 12))

entry.pack(pady=10)

btn = tk.Button(root, text="Run Command", command=process_command, bg="black", fg="white")

btn.pack()

output = tk.Label(root, text="Type a command...", font=("Arial", 12), bg="#e8f0ff")
```

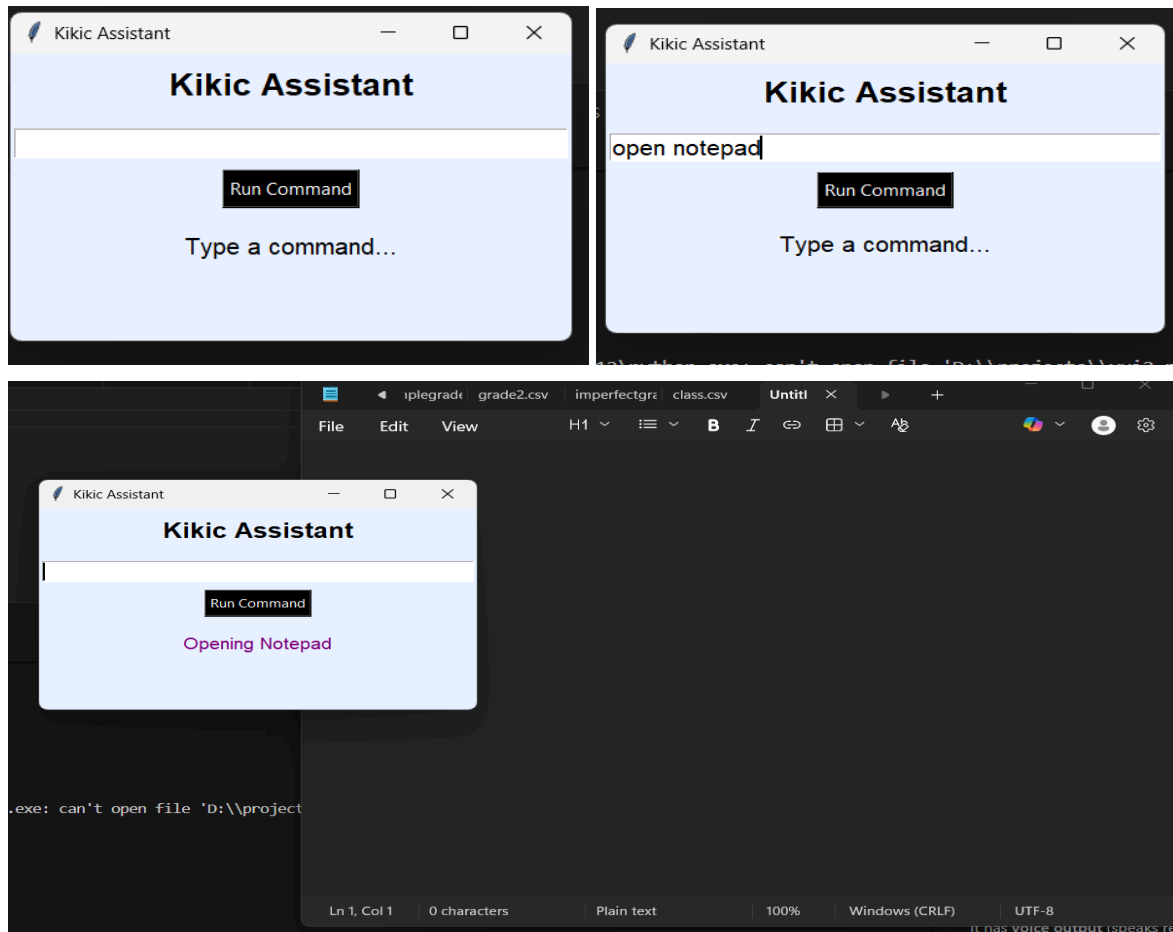


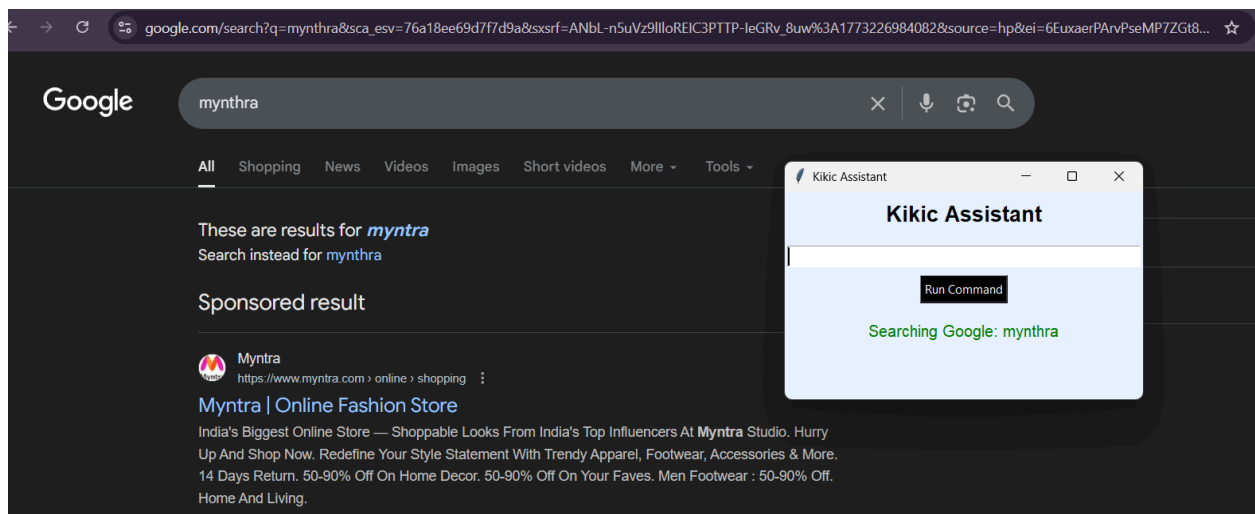
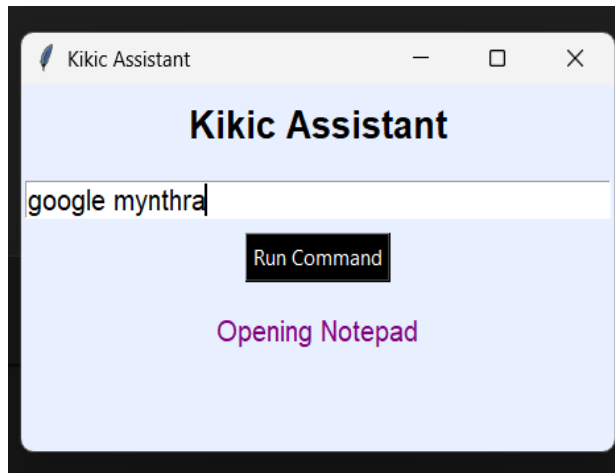
```
output.pack(pady=20)

entry.bind("<Return>", lambda event: process_command())

root.mainloop()
```

OUTPUT:





ii) VUI (Voice User Interface)

speech_recognition library for voice input and the pyttsx3 library for text-to-speech output. Make sure you have these libraries installed (pip install SpeechRecognition pyttsx3).

```
import speech_recognition as sr

import pyttsx3

import datetime

import webbrowser

import os
```

```
recognizer = sr.Recognizer()

engine = pyttsx3.init()

recognizer.pause_threshold = 1.2

def speak(text):

    engine.say(text)

    engine.runAndWait()

def process_command(text):

    text = text.lower()

    if "time" in text:

        now = datetime.datetime.now().strftime("%H:%M:%S")

        print("Current time:", now)

        speak("The time is " + now)

    elif "open youtube" in text:

        speak("Opening YouTube")

        webbrowser.open("https://youtube.com")

    elif "open google" in text:
```

```
        speak("Opening Google")

        webbrowser.open("https://google.com")

    elif "open note" in text:

        speak("Opening Notepad")

        os.system("notepad")

    elif "exit" in text:

        speak("Goodbye")

        return False

    else:

        speak("Command not recognized")

    return True

print("Start speaking... Say exit to stop.")

running = True

while running:

    with sr.Microphone() as source:

        print("Listening...")
```

```

recognizer.adjust_for_ambient_noise(source, duration=0.3)

audio = recognizer.listen(source)

try:

    text = recognizer.recognize_google(audio)

    print("You said:", text)

    running = process_command(text)

except:

    print("Not understood")

```

OUTPUT:

The screenshot shows a VS Code editor with a Python script in a file named `vui1.py`. The script imports `speech_recognition` as `sr`, `pyttsx3`, `datetime`, `webbrowser`, and `os`. It initializes a `Recognizer` object, sets a `pyttsx3` engine with a `pause_threshold` of 1.2, and sets the recognizer's `pause_threshold` to 1.2. The terminal output shows the script being executed, with prompts for listening and speaking. The output indicates that the first three attempts to recognize speech failed ("Not understood"), but the fourth attempt was successful ("You said: open YouTube"). The fifth attempt failed, and the sixth attempt was successful ("You said: open Google"). The seventh attempt failed, and the eighth attempt was successful ("You said: open Notepad"). The ninth attempt failed, and the tenth attempt was successful ("You said: open Notepad").

```

1 import speech_recognition as sr
2 import pyttsx3
3 import datetime
4 import webbrowser
5 import os
6
7 recognizer = sr.Recognizer()
8 engine = pyttsx3.init()
9         (variable) pause_threshold: float
10 recognizer.pause_threshold = 1.2

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS D:\projects> python vui1.py
Start speaking... Say exit to stop.
Listening...
Not understood
Listening...
Not understood
Listening...
You said: open YouTube
Listening...
Not understood
Listening...
You said: open Google
Listening...
Not understood
Listening...
You said: open Notepad
Listening...
Not understood
Listening...

```

Ln 1, Col 1 | 0 characters | Plain text | 100% | Windows (CRLF) | UTF-8

```
Not understood  
Listening...  
Not understood  
Listening...  
You said: let's exact  
Listening...  
You said: let's exit  
PS D:\projects> █
```